SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS: **Total Marks: 50**

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A3- Short Answer Test

Question 1 (5 Marks)
Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.
Question 2 (5 Marks)
Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.
Question 3 (5 Marks)
Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.
Question 4 (5 Marks)
Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.
Question 5 (5 Marks)
Responsive Layout: A college website must display properly on desktops, tablets, and mobile phones. How would you use CSS media queries and flexible layouts to achieve responsiveness?
Question 6(5 Marks)
CSS Styling: An online shopping website has plain product cards. How would you apply CSS colors, borders, shadows, and spacing to improve its visual appearance?
Question 7 (5 Marks)
Navigation Menu: A website navigation menu overlaps its content on smaller screens. How would you use CSS layout techniques to correct the problem?
Question 8 (5 Marks)
JavaScript Validation: A registration form accepts empty fields. How would you use JavaScript to validate the form before submission?

Question 9 (5 Marks)
Interactive Button: An e-learning website requires a button that displays a welcome message when clicked. How would you implement this functionality using JavaScript?
Question 104 (5 Marks)
. Dynamic Content: A shopping website needs to update the total price when a user changes the quantity of a product. How would you use JavaScript to calculate and display the updated price?

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING

ASSESSMENT TOOL 3 — SHORT ANSWER TEST

COURSE: INTERNET PROGRAMMING | COURSE CODE: CSA43 | CO-2

Answer Key and Worked Solutions

The following model answers cover all ten questions in the supplied assessment. Code examples are formatted for readability and can be tested in a browser by placing HTML, CSS and JavaScript in an HTML file.

Question 1 — Responsive Web Design (5 Marks)

Responsive web design (RWD) is an approach in which a website automatically adjusts its layout and content to suit different screen sizes, including mobiles, tablets and desktops. It improves readability, accessibility, navigation and user satisfaction, and avoids maintaining separate websites for each device.

Key techniques

- Use the viewport meta tag so the page uses the device width.
- Use flexible widths such as %, max-width, and CSS Flexbox or Grid instead of fixed pixel layouts.
- Use responsive images with max-width: 100% and height: auto.
- Use media queries to change columns, font sizes and navigation at suitable breakpoints.

Example

```
<!-- HTML -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<div class="cards">
  <article>Card 1</article>
  <article>Card 2</article>
  <article>Card 3</article>
</div>
```

```
/* CSS */
* { box-sizing: border-box; }
.cards {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 16px;
}
.cards article { padding: 20px; background: #eef4ff; }
@media (max-width: 768px) {
  .cards { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}
@media (max-width: 480px) {
  .cards { grid-template-columns: 1fr; }
}
```

On large screens, three cards appear in a row; on tablets, two appear; and on narrow phones, the cards stack vertically. Flexbox is useful for arranging items in one direction, while Grid is suitable for two-dimensional rows and columns.

Improvements

- Use readable font sizes, sufficient colour contrast, touch-friendly buttons and consistent spacing.
- Test at several screen widths and avoid horizontal scrolling.

Question 2 — Client-Side Form Validation (5 Marks)

Client-side validation checks user input in the browser before a form is submitted. It gives immediate feedback, prevents common input errors and reduces unnecessary server requests. JavaScript event handling can listen for the submit event, check each field, and stop submission with preventDefault() when an input is invalid.

Regular expressions (regex) describe patterns. For example, a simple email pattern can check for text before and after @ and a dot. Regex checks are useful for format validation, but they cannot prove that an email address exists.

Example

```
<form id="signupForm">
  <label for="email">Email</label>
  <input id="email" type="text" required>
  <p id="error" aria-live="polite"></p>
  <button type="submit">Register</button>
</form>
<script>
const form = document.getElementById('signupForm');
const email = document.getElementById('email');
const error = document.getElementById('error');
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

form.addEventListener('submit', function (event) {
  if (!emailPattern.test(email.value.trim())) {
    event.preventDefault();
    error.textContent = 'Enter a valid email address.';
    email.focus();
  } else {
    error.textContent = '';
  }
});
</script>
```

Usability and security

- Show a clear message beside the incorrect field and keep valid entries when another field fails.
- Use HTML attributes such as required and type="email" where appropriate.
- Repeat validation on the server; client-side checks can be bypassed. Use HTTPS and never trust user input.

Question 3 — CSS Box Model, Positioning and Media Queries (5 Marks)

The CSS box model describes how much space an element occupies. From inside to outside, it contains content, padding, border and margin. Content holds text or images; padding adds space around the content; border surrounds padding and content; and margin creates space outside the border.

```
/* Predictable sizing: width includes padding and border */
* { box-sizing: border-box; }
.panel {
  width: 100%;
  max-width: 700px;
  padding: 20px;
  border: 1px solid #b8c5d6;
  margin: 0 auto;
}
```

Positioning affects an element's placement: static is the normal flow; relative moves an element relative to its normal position; absolute positions it relative to a positioned ancestor; fixed stays relative to the viewport; and sticky behaves normally until a scroll threshold is reached.

Why layouts break and how to fix them

- Fixed widths, large images, excessive margins, long unbroken text, and absolute positioning can cause overflow or overlap.
- Use flexible widths, max-width: 100%, box-sizing: border-box, Flexbox/Grid and media queries.

```
/* Desktop default */
.layout { display: grid; grid-template-columns: 2fr 1fr; gap: 20px; }
img { max-width: 100%; height: auto; }
@media (max-width: 700px) {
  .layout { grid-template-columns: 1fr; }
}
```

The media query changes the two-column layout into one column on smaller screens. Best practices include mobile-first design, avoiding unnecessary fixed heights, using semantic HTML, checking overflow, and testing on real or emulated devices.

Question 4 — DOM and Dynamic Content (5 Marks)

The Document Object Model (DOM) represents an HTML page as a tree of objects. JavaScript can select elements, change their text or attributes, modify styles, and create or remove elements. This allows content to update without reloading the entire page.

Example

```
<p id="status">Your order is being prepared.</p>
<button id="updateBtn" type="button">Check status</button>
<script>
const statusText = document.getElementById('status');
document.getElementById('updateBtn').addEventListener('click', () => {
  statusText.textContent = 'Your order is ready for collection.';
});
</script>
```

When the button is clicked, the event listener updates the paragraph's text through textContent. The page remains in place and does not need a full reload.

Performance and user experience

- Use textContent for plain text rather than inserting untrusted input with innerHTML.
- Select elements once when practical, update only the required part of the page, and provide visible feedback.
- For data from a server, handle loading states and errors and use fetch() asynchronously when appropriate.

Question 5 — Responsive Layout (5 Marks)

I would start with a flexible, mobile-first layout. The viewport meta tag ensures the page uses the device width. Flexbox or Grid arranges content, percentages and minmax() allow flexible sizing, and media queries adjust the layout at breakpoints. Images should scale within their containers.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<div class="columns">
  <main>College announcements</main>
  <aside>Events and notices</aside>
</div>
<style>
* { box-sizing: border-box; }
.columns { display: grid; grid-template-columns: 2fr 1fr; gap: 1rem; }
.columns > * { min-width: 0; padding: 1rem; }
@media (max-width: 700px) {
  .columns { grid-template-columns: 1fr; }
}
</style>
```

On desktop, the main content and sidebar appear side by side. Below 700px, they stack vertically. I would test the website on mobile, tablet and desktop widths and ensure text and buttons remain easy to use.

Question 6 — CSS Styling for Product Cards (5 Marks)

I would give product cards a consistent background, subtle border, rounded corners, balanced spacing and a soft shadow. A clear heading, readable price and contrasting action button improve visual hierarchy. Hover effects should be subtle and should not be the only way to communicate important information.

```
<article class="product-card">
  <img src="headphones.jpg" alt="Black wireless headphones">
  <h3>Wireless Headphones</h3>
  <p class="price">INR 1,999</p>
  <button type="button">Add to cart</button>
</article>
<style>
.product-card {
  max-width: 280px; padding: 18px; background: #fff;
  border: 1px solid #e2e8f0; border-radius: 12px;
```

```
  box-shadow: 0 4px 14px rgba(0,0,0,.08);
}
.product-card img { display: block; width: 100%; height: auto; border-radius: 8px; }
.product-card h3 { margin: 14px 0 8px; color: #1f2937; }
.price { color: #0f766e; font-weight: bold; }
.product-card button { padding: 10px 14px; border: 0; border-radius: 6px;
  background: #1d4ed8; color: white; cursor: pointer; }
.product-card button:hover { background: #1e40af; }
</style>
```

Consistent spacing and colours make the cards easier to scan. I would also maintain adequate colour contrast, use meaningful image alt text and check the design on smaller screens.

Question 7 — Fixing a Navigation Menu on Small Screens (5 Marks)

The overlap may be caused by fixed widths, absolute positioning, or a menu that cannot wrap. I would keep the navigation in normal document flow, use Flexbox, allow items to wrap when suitable, and change the menu to a vertical layout on narrow screens.

```
<nav class="nav">
  <a href="#home">Home</a>
  <a href="#courses">Courses</a>
  <a href="#about">About</a>
  <a href="#contact">Contact</a>
</nav>
<style>
.nav { display: flex; flex-wrap: wrap; gap: 12px; padding: 12px;
  background: #172554; }
.nav a { color: white; text-decoration: none; padding: 8px 10px; }
@media (max-width: 600px) {
  .nav { flex-direction: column; align-items: stretch; }
  .nav a { display: block; }
}
</style>
```

The menu remains horizontal when space is available and becomes a vertical list on small screens. I would avoid unnecessary absolute positioning, ensure links have enough touch space, and test that the menu does not cover page content.

Question 8 — JavaScript Registration Form Validation (5 Marks)

JavaScript can listen for the form’s submit event and check whether required fields contain values. If a field is empty, preventDefault() stops submission, a clear error message is displayed, and focus moves to the field that needs attention.

```
<form id="registerForm">
  <label for="name">Name</label>
  <input id="name" type="text">
  <label for="password">Password</label>
  <input id="password" type="password">
  <p id="message" aria-live="polite"></p>
  <button type="submit">Create account</button>
</form>
<script>
const form = document.getElementById('registerForm');
form.addEventListener('submit', function (event) {
  const name = document.getElementById('name');
  const password = document.getElementById('password');
  const message = document.getElementById('message');

  if (name.value.trim() === '' || password.value.trim() === '') {
    event.preventDefault();
    message.textContent = 'Please complete both fields.';
    if (name.value.trim() === '') name.focus();
    else password.focus();
  } else {
    message.textContent = '';
  }
});
</script>
```

This prevents empty fields from being submitted through the form’s normal browser interaction. Real applications must also validate on the server, apply password rules, and transmit data securely over HTTPS.

Question 9 — Interactive Welcome Button (5 Marks)

An event listener can respond to a button click and update a page element with a welcome message. The DOM is used to find the button and message area.

```
<button id="welcomeBtn" type="button">Click for welcome</button>
<p id="welcomeMessage" aria-live="polite"></p>
<script>
document.getElementById('welcomeBtn').addEventListener('click', function () {
  document.getElementById('welcomeMessage').textContent =
    'Welcome to our e-learning platform! Let us start learning.';
});
</script>
```

When the user clicks the button, JavaScript displays the message immediately without reloading the page. The aria-live attribute helps assistive technologies announce the updated text.

Question 10 — Dynamic Product Total (5 Marks)

JavaScript can calculate the total by multiplying the product's unit price by the quantity selected by the user. An input event listener recalculates the total whenever the quantity changes.

```
<label for="quantity">Quantity:</label>
<input id="quantity" type="number" min="1" value="1">
<p>Total: <strong id="total">INR 499.00</strong></p>
<script>
const unitPrice = 499;
const quantityInput = document.getElementById('quantity');
const totalElement = document.getElementById('total');

function updateTotal() {
  const quantity = Math.max(1, Number(quantityInput.value) || 1);
  totalElement.textContent = (unitPrice * quantity).toLocaleString('en-IN', {
    style: 'currency', currency: 'INR'
  });
}
quantityInput.addEventListener('input', updateTotal);
updateTotal();
</script>
```

If the unit price is ■499 and the quantity is 3, the displayed total becomes INR 1,497. The minimum quantity is set to 1 and the calculation handles empty or invalid numeric input. In a real shop, the server must recalculate the final amount before payment.